@swiftgoma/auth : guide d'utilisation

Comment utiliser le client d'authentification partagé dans une application SwiftGoma. Pour savoir ce qu'est le package, ce qu'il ne fait pas et les décisions qui le justifient, lisez `docs/auth-package-scope.md` (en anglais). Version anglaise de ce guide : `docs/auth-package-usage.md`.

Dernière mise à jour : 27 octobre 2026. Branche : `feat/auth-package`.

Sommaire

1. Ce que vous obtenez
2. Installation
3. Démarrage rapide avec Expo (mobile)
4. Démarrage rapide avec Next.js et Vite (web)
5. Options de createAuth
6. Session
7. Inscription
8. Connexion
9. Vérification en deux étapes
10. Google et Apple
11. Déconnexion et utilisateur courant
12. Appeler vos propres routes d'API
13. Erreurs
14. Rendu côté serveur
15. Exigences côté serveur
16. Dépannage
17. Limites connues

---

1. Ce que vous obtenez

Le package gère **le transport, la session et l'appareil**. Votre application gère les écrans et la logique métier.

- Inscription par e-mail ou par téléphone, avec confirmation par code et renvoi du code.
- Connexion par code e-mail, code téléphone, mot de passe, Google et Apple.
- Vérification en deux étapes, et configuration obligatoire à la première connexion pour les comptes Support et Admin.
- Une session qui se renouvelle environ 2 minutes avant l'expiration du jeton d'accès, avec un filet de sécurité sur une erreur 401, pour que l'utilisateur ne voie pas d'erreur de jeton expiré.
- Un identifiant, une plateforme et un nom d'appareil envoyés à chaque inscription et connexion.
- La déconnexion (cet appareil ou tous les appareils) et l'utilisateur courant.

- Des erreurs typées, construites à partir de la réponse du serveur. Les messages sont en français et viennent du serveur sans modification.

Le même code d'application fonctionne sur mobile et sur le web. Seul l'adaptateur change.

Plateforme	Adaptateur	Jetons
Expo (Android, iOS)	`createNativeAdapter` de `@swiftgoma/auth/native`	Jeton d'accès et jeton de renouvellement dans le stockage sécurisé
Next.js, Vite	`createWebAdapter` de `@swiftgoma/auth/web`	Cookies uniquement ; JavaScript ne voit jamais un jeton

2. Installation

Le package se trouve dans `packages/auth` et est livré via les workspaces npm (pas de registre).

1. Dans le `package.json` racine, ajoutez `"apps/*"` à `workspaces` (il ne contient aujourd'hui que `"packages/*"`). Le dossier `server/` reste en dehors du workspace.
2. Dans le `package.json` de l'application, ajoutez le package et sa dépendance « peer » :

```
{
  "dependencies": {
    "@swiftgoma/auth": "*",
    "axios": "^1.7.0"
  }
}
```

3. Lancez `npm install` à la racine du dépôt.
4. Compilez le package avant que l'application l'utilise, et de nouveau après chaque modification :

```
npm run build --workspace packages/auth
```

Points d'entrée :

Import	Contenu
`@swiftgoma/auth`	`createAuth`, erreurs, constantes, types
`@swiftgoma/auth/native`	`createNativeAdapter`
`@swiftgoma/auth/web`	`createWebAdapter`, `forwardAuthHeaders`

Expo SDK 58 configure Metro pour les workspaces automatiquement et respecte le champ `exports` de `package.json`. Next.js a besoin que le package soit compilé (ou de `transpilePackages: ["@swiftgoma/auth"]`).

3. Démarrage rapide avec Expo (mobile)

Ajoutez à l'application les modules Expo dont l'adaptateur a besoin : `expo-secure-store`, `expo-crypto` et `expo-device`.

Créez un seul objet auth pour toute l'application, par exemple dans `lib/auth.ts` :

```
import * as Crypto from "expo-crypto";
import * as Device from "expo-device";
import * as SecureStore from "expo-secure-store";
import { AppState, Platform } from "react-native";
import { createAuth } from "@swiftgoma/auth";
import { createNativeAdapter } from "@swiftgoma/auth/native";

export const auth = createAuth({
  baseUrl: "https://api.swiftgoma.com",
  adapter: createNativeAdapter({
    secureStore: SecureStore,
    appState: AppState,
    os: Platform.OS,
    randomUUID: Crypto.randomUUID,
    deviceName: () => Device.modelName ?? Device.deviceName,
    userAgent: `SwiftGoma-Buyer/1.0.0 (${Platform.OS})`,
  }),
});
```

Chargez la session enregistrée une seule fois au démarrage de l'application et attendez la fin avant d'afficher le premier écran :

```
try {
  await auth.session.start();
} catch {
  // Le stockage sécurisé n'a pas pu être lu (par exemple le trousseau est verrouillé). Ce n'est
  // pas une déconnexion : réessayez plus tard au lieu d'envoyer l'utilisateur à l'écran de
connexion.
}
```

Ensuite, connectez l'utilisateur et lisez le résultat pour décider quoi afficher :

```
await auth.login.emailCode.request({ email });
const result = await auth.login.emailCode.verify({ email, code });

if (result.status === "authenticated") {
  // connecté
}
```

Options de l'adaptateur mobile

Option	Obligatoire	Signification
`secureStore`	oui	`import * as SecureStore from "expo-secure-store"`
`appState`	oui	`AppState` de React Native
`os`	oui	`Platform.OS` ; seuls `"ios"` et `"android"` sont acceptés
`randomUUID`	oui	`Crypto.randomUUID` de `expo-crypto`
`deviceName`	non	Un texte, ou une fonction qui renvoie un texte. Affiché dans la liste des appareils de l'utilisateur, coupé à 100 caractères. `Device.modelName` est plus utile que `Device.deviceName`, qui vaut un « iPhone » générique sur iOS 16 et suivants
`userAgent`	non	Un User-Agent stable, en ASCII imprimable uniquement
`keyPrefix`	non	Préfixe des clés de stockage. Par défaut `swiftgoma.auth`
`keychainService`	non	Nom du service du trousseau sur iOS
`logger`	non	Reçoit les avertissements

À savoir :

- La session est enregistrée avec l'option `AFTER_FIRST_UNLOCK`, ce qui permet à l'application de la relire quand elle démarre en arrière-plan après un premier déverrouillage du téléphone.
- Sur iOS, le trousseau peut survivre à une réinstallation : l'identifiant d'appareil peut donc rester le même. Sur Android, il change après une réinstallation.
- Une erreur de lecture du stockage sécurisé arrive dans votre code depuis `session.start()`. Elle n'est jamais traitée comme une déconnexion.
- Une session enregistrée illisible est supprimée et traitée comme une absence de session.

4. Démarrage rapide avec Next.js et Vite (web)

Créez un seul objet auth, par exemple dans `lib/auth.ts` :

```
import { createAuth } from "@swiftgoma/auth";
import { createWebAdapter } from "@swiftgoma/auth/web";

export const auth = createAuth({
  baseUrl: "https://api.swiftgoma.com",
  adapter: createWebAdapter({ deviceName: "Navigateur web" }),
});
```

Démarrez-le une fois par onglet, dans le navigateur uniquement (dans un `useEffect` de layout, ou en haut du fichier d'entrée Vite) :

```
useEffect(() => {
  void auth.session.start();
}, []);
```

Importer `lib/auth.ts` côté serveur est sans danger : l'adaptateur ne touche au navigateur que lorsqu'on l'utilise.

Options de l'adaptateur web

Option	Signification
`deviceName`	Un texte, ou une fonction qui renvoie un texte. Rien n'est envoyé par défaut
`keyPrefix`	Préfixe des clés de stockage. Commence par une lettre ou un chiffre
`logger`	Reçoit les avertissements
`environment`	Remplace les objets du navigateur (stockage, verrous, window, document). Utilisé par les tests

Ce que l'adaptateur web fait pour vous :

- **Cookies uniquement.** Les jetons vivent dans des cookies HttpOnly. Le `localStorage` du navigateur ne contient que la date d'expiration du jeton d'accès et un identifiant d'appareil.
- **Plusieurs onglets.** Une connexion ou une déconnexion dans un onglet est reprise par les autres. Un seul onglet à la fois renouvelle la session (Web Locks), et les autres adoptent le résultat sans appeler le serveur. C'est important parce que le serveur considère un ancien jeton de renouvellement rejoué après 60 secondes comme un vol.
- **Stockage bloqué.** Si le navigateur bloque `localStorage`, la session vit en mémoire seulement et un rechargement déconnecte l'utilisateur. Un avertissement est envoyé au logger.
- **Pas de User-Agent.** Les navigateurs ne permettent pas de le définir.

5. Options de createAuth

```
createAuth({ baseUrl, adapter, socials, refreshMarginSeconds, requestTimeoutMs, logger });
```

Option	Par défaut	Signification
`baseUrl`	obligatoire	**L'origine de l'API uniquement**, par exemple `https://api.swiftgoma.com`. Le package ajoute `/api/v1`. Le `http://` simple n'est accepté que pour les adresses locales et privées
`adapter`	obligatoire	L'adaptateur mobile ou web. La plateforme et le transport doivent correspondre : le web utilise les cookies, Android et iOS utilisent les jetons Bearer
`socials`	tout désactivé	`{ google: true, apple: true }`. Voir la section 10
`refreshMarginSeconds`	120	Combien de temps avant l'expiration la session se renouvelle. Un nombre entier de 10 à 600
`requestTimeoutMs`	15000	Un nombre entier de 1000 à 120000
`logger`	silencieux	Un objet avec `debug`, `warn` et `error`

Une valeur incorrecte déclenche immédiatement une `ConfigError`.

Ce que `createAuth` renvoie :

Propriété	Ce que c'est
`auth.signup`	Les parcours d'inscription (section 7, et Google et Apple en section 10)
`auth.login`	Les parcours de connexion (section 8, et Google et Apple en section 10)
`auth.twoFactor`	La seconde étape et la configuration (section 9)
`auth.logout()`, `auth.logoutAll()`, `auth.me()`	Section 11
`auth.session`	État, abonnement, renouvellement (section 6)
`auth.socials.enabled`	Les fournisseurs que cette application a activés
`auth.api`	Le client Axios configuré pour vos propres appels (section 12)
`auth.destroy()`	Arrête les minuteurs et les écouteurs. Utile dans les tests et au rechargement à chaud

6. Session

`auth.session` contient :

Membre	Signification
`state`	L'état courant (ci-dessous)
`start()`	Charge la session enregistrée. Appelez-la une seule fois et attendez-la. Renvoie l'état
`subscribe(listener)`	Appelle l'écouteur à chaque changement. Renvoie une fonction de désabonnement
`refresh()`	Renouvelle maintenant. En temps normal, vous ne l'appelez jamais
`getAccessToken()`	Le jeton d'accès courant, renouvelé d'abord s'il est sur le point d'expirer. `null` sur le web

États :

État	Signification
`{ status: "unknown" }`	`start()` n'a pas fini
`{ status: "authenticated" }`	Connecté
`{ status: "unauthenticated", reason? }`	Déconnecté. `reason` dit pourquoi la session s'est terminée

Raisons : `logout`, `session_expired`, `session_revoked`, `token_reused`, `account_suspended`. Un utilisateur qui ne s'est jamais connecté n'a pas de raison. Affichez un court message pour `account_suspended` et `token_reused` ; pour les autres, allez à l'écran de connexion.

Exemple React (aucun hook n'est fourni pour l'instant) :

```
import { useSyncExternalStore } from "react";
import { auth } from "./auth";

export function useSessionState() {
  return useSyncExternalStore(
    (onChange) => auth.session.subscribe(onChange),
    () => auth.session.state,
    () => auth.session.state,
  );
}
```

Comment fonctionne le renouvellement :

- Un minuteur renouvelle le jeton d'accès `refreshMarginSeconds` avant son expiration, avec jusqu'à 15 secondes d'écart aléatoire. Il renouvelle aussi quand l'application revient au premier plan.
- Si une requête reçoit quand même une erreur 401 pour un jeton expiré, le package renouvelle une fois et rejoue la requête. Une seconde erreur 401 est remontée à votre code.
- Une réponse hors ligne, un délai dépassé, une erreur 5xx ou 429 conservent la session et réessaient après 5, 15 et 30 secondes, puis toutes les 60 secondes.
- Un jeton de renouvellement invalide, expiré, révoqué ou réutilisé, ou un compte suspendu, met fin à la session.

7. Inscription

L'inscription se fait en deux étapes : démarrer (le serveur envoie un code), puis vérifier. Une vérification réussie connecte l'utilisateur.

```
// E-mail
await auth.signup.email.start({
  name: "Aline Kabila",
  email: "aline@example.com",
  role: "BUYER", // "BUYER" | "SELLER" | "RIDER"
  acceptTerms: true,
  confirmAge: true,
});
const { user } = await auth.signup.email.verify({ email: "aline@example.com", code: "123456" });

// Téléphone
await auth.signup.phone.start({ name, phone: "+243810000000", role, acceptTerms: true,
confirmAge: true });
await auth.signup.phone.verify({ phone: "+243810000000", code });

// Demander un nouveau code
await auth.signup.email.resend({ email });
await auth.signup.phone.resend({ phone });
```

- `start` renvoie `{ method, target, expiresMinutes }` : où le code est parti (`"EMAIL"` ou `"PHONE"`) et combien de temps il reste valable.
- `acceptTerms` et `confirmAge` doivent valoir exactement `true`. Affichez les cases à cocher dans votre interface.
- `resend` renvoie `{ message }`.

8. Connexion

Toute connexion se termine par l'un de ces trois résultats :

`result.status`	Signification	Étape suivante
`authenticated`	Connecté. Contient `user` et `isNewDevice`	Aller dans l'application
`two_factor_required`	Le compte demande sa seconde étape	Section 9, `twoFactor.verify`
`two_factor_enrollment_required`	Un compte Support ou Admin doit d'abord configurer la double vérification	Section 9, `twoFactor.enroll`

Les deux résultats « deux étapes » contiennent `challengeExpiresIn` (en secondes, environ 5 minutes). Le jeton de la seconde étape reste à l'intérieur du package : vous ne le manipulez jamais.

```
// Code e-mail
await auth.login.emailCode.request({ email });
const result = await auth.login.emailCode.verify({ email, code });

// Code téléphone. `channel: "EMAIL"` envoie le code à l'e-mail du compte au lieu d'un SMS.
await auth.login.phoneCode.request({ phone, channel: "EMAIL" });
const result2 = await auth.login.phoneCode.verify({ phone, code });

// Mot de passe
const result3 = await auth.login.password({ email, password });
```

Traitez le résultat à un seul endroit :

```
switch (result.status) {
  case "authenticated":
    return goToApp(result.user);
  case "two_factor_required":
    return goToSecondStep();
  case "two_factor_enrollment_required":
    return goToEnrollment();
}
```

La connexion se fait par e-mail pour le code e-mail et le mot de passe, et par téléphone pour le code téléphone. Le mot de passe oublié et la récupération de compte ne font pas partie de ce package.

9. Vérification en deux étapes

Seconde étape après `two_factor_required`. Envoyez le code de l'application ou un code de secours :

```
await auth.twoFactor.verify({ code: "123456" });
// ou
await auth.twoFactor.verify({ backupCode: "ABCD-EFGH" });
```

En cas de succès, l'utilisateur est connecté. Un code erroné déclenche une erreur et garde l'étape en attente : l'utilisateur peut réessayer jusqu'à la fin de `challengeExpiresIn`.

Configuration après `two_factor_enrollment_required` (première connexion d'un compte Support ou Admin) :

```
const setup = await auth.twoFactor.enroll.start();
// setup.secret       le secret sous forme de texte
// setup.otpauthUri   l'adresse à afficher en code QR dans l'application d'authentification

const done = await auth.twoFactor.enroll.confirm({ code: "123456" });
// done.backupCodes   affichés UNE SEULE FOIS : montrez-les à l'utilisateur tout de suite
```

- `enroll.confirm` connecte l'utilisateur et renvoie `{ user, isNewDevice, backupCodes }`.

- Le package n'enregistre ni ne journalise jamais les codes de secours. Si votre application les perd avant de les afficher, ils sont perdus.
- Appeler la mauvaise étape (par exemple `verify` pendant la configuration, ou l'un ou l'autre sans étape en attente) déclenche une erreur sans envoyer de requête. Le code d'erreur indique le cas (`TWO_FACTOR_SETUP_REQUIRED`, `TWO_FACTOR_ALREADY_ENABLED`, `TWO_FACTOR_CHALLENGE_EXPIRED`).

10. Google et Apple

La connexion sociale est **désactivée par défaut**. Activez-la application par application :

```
createAuth({ /* ... */, socials: { google: true, apple: true } });
```

- `auth.socials.enabled` liste ce que votre écran de connexion peut afficher, par exemple `["google"]`. Le package ne dessine aucun bouton.
- Un fournisseur désactivé déclenche `SocialLoginDisabledError` (code `SOCIAL_LOGIN_DISABLED`) avant l'envoi de toute requête.
- Les bibliothèques Google et Apple restent dans votre application, car elles ont besoin des identifiants client propres à l'application.

Vous donnez au package une fonction qui renvoie le jeton d'identité du fournisseur. Le package demande au serveur un nonce à usage unique, appelle votre fonction avec ce nonce, puis envoie le jeton et le nonce :

```
const getIdToken = async (nonce: string): Promise<string> => {
  // Appelez ici votre bibliothèque Google ou Apple et passez `nonce` exactement tel quel.
  // Ne le hachez pas : le serveur le compare tel quel.
  return idTokenFromTheProvider;
};

// Connexion
await auth.login.google({ getIdToken });
await auth.login.apple({ getIdToken });

// Inscription
await auth.signup.google({ getIdToken, role: "BUYER", acceptTerms: true, confirmAge: true });
await auth.signup.apple({
  getIdToken,
  role: "BUYER",
  acceptTerms: true,
  confirmAge: true,
  name: "Aline Kabila", // facultatif : Apple ne donne le nom que la première fois
});
```

Les quatre renvoient le même résultat de connexion que la section 8.

- Si l'utilisateur annule dans l'écran du fournisseur, l'erreur propre à la bibliothèque arrive dans votre code sans modification. Aucune requête de connexion n'est envoyée.

- Une fonction qui renvoie un jeton vide déclenche une `TypeError`.
- Le nonce est à usage unique et valable 10 minutes.

À savoir :

- **Règle 4.8 de l'App Store.** Si une application iOS propose Google (ou une autre connexion tierce) comme principal moyen de se connecter, Apple demande une option équivalente qui limite les données au nom et à l'e-mail, permet de masquer l'e-mail et ne suit pas l'utilisateur à des fins publicitaires. Se connecter avec Apple est le choix habituel. Le package écrit un avertissement dans le logger quand une version iOS active Google sans Apple. La règle a des exceptions (par exemple les applications qui n'utilisent que leur propre connexion) ; lisez la formulation actuelle avant de décider.
- **Relais privé d'Apple.** Un utilisateur inscrit avec un vrai e-mail qui utilise ensuite l'e-mail masqué d'Apple obtient un compte séparé. Les comptes ne sont liés que sur un e-mail vérifié identique. Expliquez-le sur votre écran de connexion.
- **Apple sur Android et sur le web** demande le parcours web d'Apple (un Services ID et une page de redirection). Certaines applications peuvent laisser Apple désactivé à cet endroit.

11. Déconnexion et utilisateur courant

```
const { serverRevoked } = await auth.logout();
const { revoked } = await auth.logoutAll();
const me = await auth.me();
```

- `logout()` déconnecte **toujours** cet appareil et ne déclenche jamais d'erreur. `serverRevoked` vaut `false` quand le serveur n'a pas pu être joint ; vous pouvez en informer l'utilisateur.
- `logoutAll()` déconnecte tous les appareils et renvoie le nombre de sessions terminées. Si l'appel au serveur échoue, elle **déclenche une erreur et garde la session** pour que l'utilisateur puisse réessayer.
- `me()` renvoie l'utilisateur courant : id, name, email, phone, role, status, `emailVerified`, `phoneVerified`, `hasPassword`, `twoFactorEnabled`, `verificationStatus`, `createdAt`, `lastLoginAt` et `sessionId`.
- `me()` et `logoutAll()` déclenchent `NotAuthenticatedError` quand personne n'est connecté.

12. Appeler vos propres routes d'API

`auth.api` est le client Axios que le package utilise. Servez-vous-en pour les appels propres à l'application afin d'obtenir le même comportement :

```
const response = await auth.api.get("/shops/mine"); //
https://api.swiftgoma.com/api/v1/shops/mine
```

- Les chemins sont relatifs à `<origine>/api/v1`.
- Sur mobile, l'en-tête `Authorization: Bearer` est ajouté pour vous. Sur le web, les cookies sont envoyés (`withCredentials`) et aucun en-tête n'est ajouté.
- Un jeton expiré est renouvelé et la requête est rejouée une fois.
- Chaque échec devient une erreur typée (section 13). Une requête annulée passe sans modification.

- Ajoutez `skipAuth: true` à la configuration d'une requête pour un appel qui ne doit pas porter de jeton.

13. Erreurs

Chaque échec est une `ApiError` ou l'une de ses sous-classes. Lisez `error.code` pour décider quoi faire, et affichez `error.message` (en français, venant du serveur) à l'utilisateur.

Classe	Quand
`ApiError`	Classe de base. Contient `code`, `status`, `message`, `details`, `requestId` et l'erreur Axios d'origine dans `cause`
`UnauthorizedError`	401
`ForbiddenError`	403
`NotFoundError`	404
`ConflictError`	409
`ValidationError`	422. `error.fields` est une liste de `{ field, message }`
`TooManyRequestsError`	429. `error.retryAfterSeconds` peut être renseigné
`ServerError`	5xx
`NetworkError`	Le serveur n'a pas pu être joint
`TimeoutError`	Le serveur a mis trop de temps
`NotAuthenticatedError`	Vous avez demandé quelque chose qui exige une session (`NOT_AUTHENTICATED`)
`SocialLoginDisabledError`	Un fournisseur que l'application n'a pas activé

```
import { hasErrorCode, isApiError } from "@swiftgoma/auth";

try {
  await auth.login.password({ email, password });
} catch (error) {
  if (hasErrorCode(error, "INVALID_CREDENTIALS")) return showWrongPassword();
  if (isApiError(error)) return showMessage(error.message);
  throw error;
}
```

Les codes que vous rencontrerez le plus souvent : `INVALID_CREDENTIALS`, `OTP_INVALID`, `OTP_COOLDOWN`, `OTP_HOURLY_LIMIT`, `OTP_ATTEMPTS_EXCEEDED`, `ACCOUNT_EXISTS`, `ACCOUNT_NOT_FOUND`, `ACCOUNT_SUSPENDED`, `ACCOUNT_PENDING_APPROVAL`, `TWO_FACTOR_INVALID`, `TWO_FACTOR_LOCKED`, `TWO_FACTOR_CHALLENGE_EXPIRED`, `GOOGLE_TOKEN_INVALID`, `APPLE_TOKEN_INVALID`, `SESSION_REVOKED`, `NETWORK_ERROR`, `TIMEOUT`. La liste complète est `ERROR_CODES` dans le package.

14. Rendu côté serveur

Pour une page Next.js qui doit lire l'utilisateur connecté côté serveur, transmettez à l'API les cookies de session du visiteur :

```
import { headers } from "next/headers";
import { forwardAuthHeaders } from "@swiftgoma/auth/web";
import { auth } from "@/lib/auth";

const cookie = (await headers()).get("cookie");
const response = await auth.api.get("/auth/me", {
  headers: forwardAuthHeaders({ cookie, origin: "https://app.swiftgoma.com" }),
});
```

- Seuls les deux cookies de session (`__Host-swg_access` et `swg_refresh`) sont transmis ; tous les autres cookies sont écartés. Sans cookie de session, aucun en-tête n'est ajouté.
- `origin` est l'origine de votre application web. L'API en a besoin pour les écritures authentifiées par cookie.
- **Règle d'hôte.** Le cookie d'accès porte le préfixe `__Host-` , il appartient donc à un seul hôte. Cela ne fonctionne que si le navigateur et votre serveur Next.js joignent l'API sous le même hôte, par exemple via une réécriture ou un reverse proxy. Si l'API est sur un autre hôte, appelez-la depuis le navigateur.
- Un renouvellement fait côté serveur ne peut pas définir les cookies du visiteur. Gardez les renouvellements dans le navigateur et utilisez cet utilitaire pour les lectures.

15. Exigences côté serveur

Le package suppose que l'API SwiftGoma (`server/`) est configurée ainsi :

- Chaque origine d'application web figure dans `CORS_ORIGINS` .
- L'application web et l'API partagent un domaine enregistrable (les cookies sont `SameSite=Lax`), ou l'application web appelle l'API par son propre serveur.
- Google et Apple sont configurés côté serveur (`GOOGLE_CLIENT_IDS` , `APPLE_CLIENT_IDS`) avant de les activer dans une application.
- Changement serveur en attente : les réponses de connexion, de vérification et de configuration devraient renvoyer `expiresIn` . D'ici là, le web suppose un jeton d'accès de 15 minutes (voir la section 17).

16. Dépannage

Symptôme	Cause probable et correction
`ConfigError: baseUrl must be the API origin only`	Retirez le chemin de `baseUrl` ; le package ajoute `/api/v1`
`ConfigError: baseUrl must use https`	Le `http://` simple ne fonctionne que pour localhost et les adresses privées
`ConfigError` à propos du transport	Le web doit utiliser l'adaptateur web, Android et iOS l'adaptateur mobile
Chaque renouvellement web échoue et recommence chaque minute	L'origine de l'application manque dans `CORS_ORIGINS` du serveur (`CORS_NOT_ALLOWED`)
Les utilisateurs web sont déconnectés à chaque rechargement	Le navigateur bloque `localStorage` ; cherchez l'avertissement de l'adaptateur dans le logger
`session.start()` déclenche une erreur sur mobile	Le stockage sécurisé n'a pas pu être lu. Réessayez plus tard ; ne le traitez pas comme une déconnexion
Un bouton social ne fait rien	Le fournisseur est désactivé : vérifiez `auth.socials.enabled`
`SESSION_REVOKED` après l'usage d'un autre appareil	L'utilisateur s'est déconnecté partout, ou un jeton volé a été détecté
Next.js ne trouve pas le package	Compilez-le d'abord, ou ajoutez `transpilePackages: ["@swiftgoma/auth"]`

17. Limites connues

- **Renouvellement anticipé sur le web.** Les réponses de connexion web du serveur ne contiennent pas encore `expiresIn` : le web suppose donc 15 minutes et découvre une expiration plus précoce grâce au filet de sécurité du 401. Sur mobile, le package lit l'expiration du jeton lui-même et renouvelle à temps.
- **Le User-Agent sur un vrai appareil** n'a pas encore été vérifié. Il faut pour cela la première application Expo.
- **Les tests de contrat** qui font tourner le package contre le vrai serveur existent mais n'ont pas encore été exécutés. Les tests unitaires utilisent de faux serveurs.
- **Hors du package :** les passkeys, le mot de passe oublié, la récupération de compte, le lien d'un compte Google ou Apple à un compte existant, les écrans de profil et de sécurité, l'enregistrement des jetons de notification push, toute interface, les traductions. Des hooks React pourraient venir plus tard.
- tsdown est en version antérieure à 1.0 et TypeScript est figé à `~6.0.3` : attendez-vous à de petits changements d'options de compilation.